

UNIVERSIDAD DE MONTERREY
Escuela de Ingeniería y Tecnología
Ingeniería en Tecnologías Computacionales

Decisiones Arquitectónicas

Autor(es)
600046 Fernando Olivares Del Valle

Profesor (a)
Prof. Raúl Morales Salcedo

Doy mi palabra que he realizado esta actividad con integridad académica

San Pedro Garza García, N.L. 27 de Agosto, 2026

Decisiones arquitectónicas:

Para el desarrollo de la aplicación web se llevaron a cabo las siguientes decisiones arquitectónicas con el objetivo de cumplir con los requerimientos funcionales y no funcionales y entregar una aplicación web funcional.

Decisión 1: Arquitectura monolítica server-side

Se decidió desarrollar la aplicación web utilizando una arquitectura monolítica server-side con Node.js, Express y EJS.

Beneficios: Entre los beneficios que utiliza este tipo de arquitectura se incluye que es solo un proceso desplegable, reduciendo la complejidad de la infraestructura, puesto que no representa ninguna necesidad de utilizar múltiples microservicios para su funcionamiento. Además, el despliegue es más sencillo, y es adecuado para el tamaño del servicio desarrollado tanto para el equipo computacional como el volumen de datos.

Limitaciones: El principal problema que representa esta aplicación web es su escalabilidad en el futuro para un mayor volumen de datos o más recursos computacionales. Esto se debe a que los módulos comparten el mismo proceso para su funcionalidad y despliegue. Existe gran dependencia entre los módulos del servidor, lo que afecta su disponibilidad.

Condición futura: La condición futura que implicaría realizar cambios, sería que el volumen de usuarios concurrentes o de datos creciera de manera significativa y los recursos no son suficientes para sostener los servicios del sistema. También la necesidad de escalabilidad implicaría realizar cambios al despliegue y migrar la arquitectura hacia una de microservicios.

Decisión 2: Acceso directo a PostgreSQL

Beneficios: Entre los beneficios que representa el acceso directo a PostgreSQL es la seguridad, la optimización del rendimiento y la limpieza del código. Al tratar los datos como valores y no como código ejecutable mitiga por completo la posibilidad de fallas de seguridad. A su misma vez, PostgreSQL planifica y optimiza las consultas para su reutilización disminuyendo el consumo de recursos.

Limitaciones: Entre las limitaciones se encuentra que el escribir SQL de manera manual se vuelve repetitivo, ya que se tienen que redactar a mano para cada tabla, a diferencia de otras tecnologías como ORM, que las consultas se generan de forma automática a partir de un modelo definido.

Condición futura: En el caso de que se tuviera que escalar la base de datos, con un mayor número de tablas convendría optar por un ORM con el objetivo de reducir el código repetitivo, y así facilitando el mantenimiento.

Decisión 3: Renderizado server-side utilizando EJS

Beneficios: Entre los beneficios que representa utilizar EJS es que permite combinar HTML con JavaScript dentro de las plantillas creadas, facilitando la inserción de datos dinámicos sin la necesidad de hacer HTMLs de manera manual. Además el HTML llega directamente y completo al navegador sin utilizar JSON y XML.

Limitaciones: Entre las limitaciones que implica este renderizado es que, cada acción del usuario implica recargar la página por completo. Haciendo que la experiencia del usuario no sea la mejor.

Condición futura: Con el objetivo de tener una mejor experiencia para el usuario final, sería buena opción utilizar una API que devuelva datos con JSON, y un frontend que se encargue de entregar la interfaz al navegador de manera dinámica.